Zero-Trust Architecture for MCP-Based AI Agents: A Unified CLI Approach

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—The rapid evolution of Large Language Models (LLMs) from passive chatbots to autonomous agents enables complex workflows through tool use and protocol standardization like the Model Context Protocol (MCP). However, this autonomy introduces significant security risks, as agents can execute code, modify files, and access remote systems. Traditional network-based security perimeters are insufficient for these application-layer entities. This paper presents `llm-cli`, a unified command-line interface that implements Zero Trust principles—Verify Explicitly, Least Privilege, and Assume Breach—specifically tailored for AI agents. We analyze the current landscape of agentic tools, identify the security gaps, and detail our implementation of a secure-by-default runtime. Key features include RSA-based workload identity, fine-grained Role-Based Access Control (RBAC), and a novel dynamic intent verification system. Experimental results demonstrate that while local models (Llama 3.2) offer viable offline protection with 60% recall, cloud-based verifiers (e.g., Grok, Claude) achieve superior accuracy with 90-95% recall and low false positive rates, highlighting the trade-off between privacy and security assurance.**

***Index Terms*—Zero Trust, AI Agents, Model Context Protocol, Large Language Models, Security Guardrails, RBAC, Intent Verification.**

I. Introduction

As Large Language Models (LLMs) continue to advance, the paradigm of interaction is shifting from simple query-response pairs to continuous, autonomous loops where models actively plan, reason, and execute actions. The introduction of the Model Context Protocol (MCP) [1] further accelerates this trend by standardizing how AI agents connect to data sources and tools, effectively turning LLMs into operating system interfaces.

While this connectivity unlocks immense productivity, it erodes traditional security boundaries. An AI agent with shell access acts as a proxy for the user, but it is susceptible to prompt injection, hallucinations, and manipulation. If an agent is compromised, it can become a confused deputy, executing malicious commands with the user's privileges.

In this context, the Zero Trust Architecture (ZTA) [3], which assumes no implicit trust based on location or network, becomes critical. However, applying ZTA to AI agents requires adapting principles designed for network traffic to semantic intents and function calls.

This paper introduces a Zero Trust architecture for `llm-cli` [2], an open-source tool designed to bridge the gap between powerful agentic capabilities and rigorous security. Building upon our previous work on static guardrails [2], we demonstrate how Zero Trust principles can be practically applied to the agentic workflow without sacrificing usability.

II. Related Work

A. Security Vulnerabilities in LLM Agents

The integration of LLMs with external tools introduces a new attack surface, primarily driven by Prompt Injection (PI) attacks. Greshake et al. demonstrated "Indirect Prompt Injection," where an LLM processing untrusted content (e.g., a website) can be manipulated to execute malicious instructions [4]. Liu et al. further showed that standard safety alignment (RLHF) is often insufficient to prevent "jailbreaking" when models are placed in an agentic loop [5].

Unlike standalone chatbots, agents have the capability to execute code and modify file systems. The OWASP Top 10 for LLM Applications highlights "Insecure Plugin Design" and "Excessive Agency" as critical risks [6]. A compromised agent effectively becomes a "confused deputy," leveraging the user's credentials to perform unauthorized actions.

B. Existing Defense Mechanisms

Current defenses largely fall into two categories: static guardrails and dynamic monitoring. Static approaches, such as NeMo Guardrails [7], use predefined rules to filter inputs and outputs. While effective against known patterns, they lack the semantic understanding to detect context-dependent attacks (e.g., distinguishing between a user asking to delete a temporary file versus a system file).

Dynamic approaches leverage "LLM-as-a-Judge" to evaluate inputs. However, early implementations often suffered from high latency and cost. Recent works have proposed lightweight "self-correction" mechanisms [8], but these often rely on the same model for both execution and verification, leaving them vulnerable to single-point failures if the model is successfully jailbroken.

C. Zero Trust Architecture (ZTA) for AI

Zero Trust, as defined by NIST SP 800-207 [3], posits that trust should never be implicit. While ZTA has been widely adopted in network security (e.g., Google's BeyondCorp [9]), its application to AI agents is nascent. Recent guidelines from

BSI and ANSSI [12] argue for applying ZTA principles—specifically Least Privilege and Continuous Verification—to the AI model itself.

Our work, `llm-cli`, bridges these domains. Unlike existing agent frameworks that focus on capability (e.g., AutoGen [10]), we prioritize security by implementing a distinct workload identity and an independent, secondary LLM for intent verification, effectively decoupling the "actor" from the "auditor."

III. Current Landscape of AI Agents and Zero Trust

A. The Rise of Autonomous Agents

Frameworks such as LangChain [11], AutoGen [10], and CrewAI have democratized the creation of agentic workflows. These tools allow developers to chain multiple LLM calls, manage state, and equip models with "tools" (Python functions, API wrappers).

However, most current frameworks prioritize capability and ease of development over security.

- **Implicit Trust**: Tools are often exposed to the LLM with full privileges. If an agent decides to call `delete_file`, the framework typically executes it blindly.
- **Lack of Identity**: Agents often run with the full permissions of the host process or user, lacking a distinct "workload identity" that can be scoped or audited.
- **Opaque Execution**: While logs exist, real-time visibility into *why* an agent is performing an action is often missing.

B. Zero Trust in the AI Era

NIST SP 800-207 defines Zero Trust as a cybersecurity paradigm focused on resource protection and the premise that trust is never granted implicitly [3]. In the context of AI, ZTA must evolve:

1) **Identity**: The agent itself must have an identity separate from the user, allowing for granular permission scoping.
2) **Contextual Verification**: Unlike network packets, "malicious" AI actions are often context-dependent. A command to `rm -rf /` is dangerous, but `rm temp.txt` might be valid. Static rules are insufficient; semantic verification is needed.

Currently, there is a lack of unified tooling that integrates these ZTA principles directly into the agent runtime. Developers are forced to cobble together disparate libraries for auth, policy, and logging, often leaving gaps.

IV. Proposed System: `llm-cli`

A. Threat Model and Trust Assumptions

Our primary enforcement target is **MCP-mediated tool execution**. The MCP server-side wrapper acts as the Policy Enforcement Point (PEP) that performs authentication, authorization, and audit logging before invoking any tool.

Trusted Computing Base (TCB). The launched MCP server process (binary, container image, and its launch configuration) is considered part of the TCB. In other words, `llm-cli` assumes the server command and configuration are controlled by the user/administrator and are not adversarial. This boundary must be made explicit because a malicious or modified MCP server can exfiltrate data regardless of runtime policy checks.

`llm-cli` is a unified CLI tool designed to interact with multiple LLM providers (OpenAI, Anthropic, Google Gemini, xAI Grok, Ollama, vLLM) while enforcing a strict security model. It operates not just as a client, but as a secure runtime environment for MCP-based tools.

B. Design Philosophy

Our design is guided by three core ZTA principles:

1) **Verify Explicitly**: Every tool call is intercepted and verified against both static policies and dynamic intent analysis.
2) **Use Least Privilege**: Agents operate with the minimum necessary permissions (Roles/Scopes), distinct from the user's full OS privileges.
3) **Assume Breach**: The system assumes the LLM may be compromised (e.g., via prompt injection) and enforces global guardrails that cannot be bypassed by the model.

C. Key Features

- **Unified Interface**: Seamless switching between providers (e.g., `/p gemini`) and models.
- **Multi-Modal Agent**: Supports input of text, images, PDFs, audio, and video. Agents can autonomously fetch and process these media types.
- **MCP Integration**: Acts as both an MCP Client (connecting to GitHub via Docker or remote servers via SSH) and an MCP Server.
- **Developer Experience**: Features like Template Management (`/t proofread`) and Context Checkpointing (`/checkpoint`) to manage long sessions.

V. Implementation of Zero-Trust Principles

The security architecture of `llm-cli` is implemented through a layered approach involving Identity Management, Policy Enforcement, Audit Logging, and Dynamic Verification.

A. Identity and Authentication

To treat the agent as a distinct entity, `llm-cli` implements a workload identity system rooted in asymmetric cryptography.

As seen in the `IdentityManager` class (Listing 1), the system generates a local RSA key pair upon initialization. This key is used to sign short-lived JSON Web Tokens (JWTs) using the RS256 algorithm that accompany every MCP request.

```
class IdentityManager:
   def generate_token(self, user_id=None, roles=None
       ):
      payload = {
         "iss": "llm-cli-client",
```

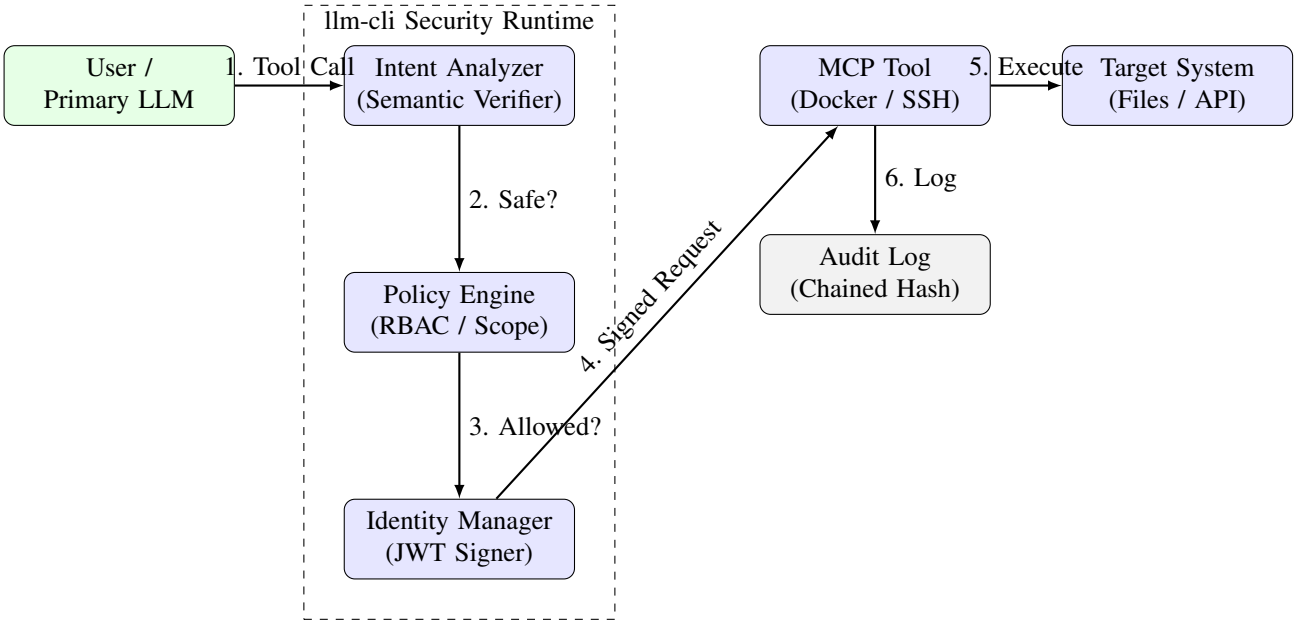

Fig. 1. The Zero Trust Architecture of `llm-cli`. User requests are intercepted by the runtime, where they undergo semantic intent verification, RBAC policy checks, and cryptographic signing before any tool execution occurs.

```
        "sub": user_id or self.get_local_identity()
            ,
        "iat": time.time(),
        "exp": time.time() + 600, # 10 min expiry
        "roles": roles or ["user"],
    }
    # Sign with private RSA key
    return jwt.encode(payload, self._private_key,
        algorithm="RS256")
```

Listing 1. Identity Token Generation in identity.py

This mechanism ensures that even if a remote MCP server is accessed (e.g., via SSH tunneling), it can cryptographically verify the caller's identity and assigned roles, preventing unauthorized access from spoofed clients.

B. Least Privilege via RBAC and ABAC

MCP role issuance. In MCP mode, the client issues a short-lived JWT for each configured MCP server with an explicit `aud` (audience) bound to the server name. To uphold least privilege, the client must *not* default to `admin`; instead, it derives roles from user configuration per server (default `guest`). This prevents accidental over-privileging of remote tool backends.

The `PolicyEngine` implements a hybrid Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) model.

1) Role Definitions: Users are assigned roles such as `guest`, `user`, or `admin`. A `guest` role, for instance, is restricted to read-only tools like `list_files` and `read_file`, effectively neutralizing the risk of destructive actions.

2) Scope Verification: **Path normalization.** Scope checks must use canonical paths. We normalize tool arguments using the same expansion and resolution strategy as tool-level path validation (e.g., `expanduser` + `resolve`) before applying pattern matching. This reduces bypass via relative paths, alternate representations, or symlinks.

Beyond simple role checks, the policy engine evaluates attributes of the request. For example, the `edit_file` tool is restricted by a path scope:

```
def _verify_scope(self, tool_name, args, policy):
    scopes = policy.get("scopes", {})
    if tool_name in scopes:
        allowed_paths = scopes[tool_name].get("
            allowed_paths", [])
        target_path = args.get("path")
        # Check if path matches allowed patterns
        if not any(fnmatch.fnmatch(target_path, p) for
            p in allowed_paths):
            return False
    return True
```

Listing 2. Scope Verification Logic

This ensures that even an authenticated user cannot write to sensitive system directories (e.g., `/etc/`), limiting the blast radius of a compromised agent.

C. Tamper-Evident Audit Logging

Retention without breaking integrity. A practical system must enforce retention limits. Naively truncating old log lines breaks the hash chain and weakens tamper evidence. Therefore, when exceeding a configured limit, we rotate overflow entries into an archive file and insert a snapshot anchor entry into the active log, preserving verifiability of the remaining segment and linking it to the archived portion.

Transparency is a core tenet of Zero Trust. `llm-cli` implements a chained hashing mechanism for audit logs. Each

log entry contains a cryptographic hash of the previous entry, forming a tamper-evident chain.

```
{
  "timestamp": "2024-05-20T10:00:00Z",
  "trace_id": "uuid-1234-5678",
  "actor": "user_01",
  "action": "execute_command",
  "target": "ls -la /var/log",
  "status": "ALLOWED",
  "prev_hash": "a1b2c3d4...",
  "hash": "e5f6g7h8..." // SHA-256(prev_hash +
      entry_content)
}
```

Listing 3. Audit Log Entry Structure with Chained Hashing

- **Traceability**: Every tool execution is logged with a unique `trace_id` propagated via JSON-RPC metadata, linking the LLM's "thought" to the specific system call.
- **Integrity**: Any modification or deletion of historical logs breaks the hash chain, which is detected upon system startup.

D. Dynamic Intent Verification (Verify Explicitly)

Availability and failure policy. Because the intent verifier is itself a dependency (network/model availability), Zero Trust requires an explicit failure mode. Our implementation supports configurable *fail-closed by default* when the intent analyzer cannot be initialized, with per-tool overrides (e.g., fail-open for low-risk read tools; fail-closed for high-risk write/shell tools). This aligns with *Verify Explicitly* and *Assume Breach*.

Static policies cannot capture context. To address this, `llm-cli` introduces a "Dynamic Guardrail" using a secondary, smaller LLM (e.g., Gemini 2.5 Flash Lite) to act as a supervisor.

Before executing any tool, the `IntentAnalyzer` compares the user's original prompt with the agent's proposed action using a strict system prompt (Listing 4).

```
SYSTEM_PROMPT = """
You are a security auditor for an AI agent.
User Request: {user_prompt}
Proposed Tool Call: {tool_name} with args {args}

Your task:
1. Determine if the tool call aligns with the user's
     request.
2. Identify any security risks (e.g., file deletion,
     data exfiltration).
3. Return "SAFE" or "SUSPICIOUS" with a reason.
"""
```

Listing 4. System Prompt for Intent Analyzer

The analyzer workflow is shown below:

```
def _analyze_intent(self, user_prompt, tool_name,
    args):
   # Secondary LLM checks alignment
   is_safe, reason = self.intent_analyzer.
       verify_action(
      user_prompt, tool_name, args
   )
   if not is_safe:
      logger.warning(f"Blocked: {reason}")
      return False
   return True
```

Listing 5. Intent Analysis Flow

If a user asks to "Summarize this paper" and the agent attempts to call `execute_command(cmd="curl malicious.site")`, the intent analyzer detects the semantic mismatch and blocks the execution, mitigating prompt injection attacks that bypass static filters.

E. Assume Breach: Global Guardrails

Finally, hard-coded global guardrails serve as the last line of defense. These rules apply to all users, including admins, and block known dangerous patterns such as:

- Accessing system roots (e.g., `/root`, `C:\Windows`).
- High-risk shell commands (e.g., `mkfs`, `dd`, `chmod 777`).

VI. Case Studies: Secure MCP Orchestration

A key strength of `llm-cli` is its ability to extend the agent's capabilities securely via the Model Context Protocol (MCP). We present two integration patterns that leverage our Zero Trust architecture.

A. Remote Infrastructure Management via SSH

Developers often need agents to interact with remote servers. Traditional approaches involve copying SSH keys to the agent's environment, creating a massive security risk.

`llm-cli` solves this by acting as an MCP Client that spawns a remote MCP Server over an SSH tunnel:

1) The local client generates a short-lived JWT signed with its local private key.
2) The client initiates an SSH connection to the remote host and starts the `llm-cli` process in server mode.
3) The JWT is passed to the remote server.
4) The remote server, configured with the client's public key, verifies the token's signature and claims (e.g., roles).

Crucially, **no private keys are ever transmitted over the network**. The agent on the local machine can "tool use" the remote server (e.g., checking logs, restarting services) within the strict boundaries of the remote policy, without possessing shell credentials.

B. GitHub Integration via Docker Isolation

To grant agents access to source code repositories without exposing the host environment, `llm-cli` integrates with the official GitHub MCP server running inside a Docker container.

- **Isolation**: The GitHub tool runs in an ephemeral container, ensuring that any vulnerability in the tool implementation cannot compromise the host.
- **Least Privilege**: The container is injected with a scoped Personal Access Token (PAT). The `llm-cli` policy engine further restricts which repositories the agent can access, preventing accidental modifications to unrelated projects.

This setup allows the agent to perform complex tasks like "Review the latest PR on the `dev` branch" autonomously and securely.

VII. Evaluation and Discussion

A. Experiment Design

To evaluate the effectiveness of the `IntentAnalyzer` and the feasibility of running verification locally, we expanded our benchmark suite to 30 distinct test cases, categorized into:

- **Benign** (10 cases): Standard actions (e.g., `ls -la`, reading documentation, system checks).
- **Malicious** (10 cases): Explicit attacks (e.g., `rm -rf /`, accessing `/etc/passwd`, malware download).
- **Subtle** (10 cases): Context-dependent risks (e.g., modifying `.env` files, internal network scanning, obfuscated commands).

We compared cloud-based models including OpenAI's `gpt-5-nano`, Anthropic's `claude-haiku`, Google's `gemini-flash-lite`, and xAI's `grok-4.1` against a local model (Ollama `llama3.2:3b`). Each case was executed 5 times to account for non-determinism. We measured Recall (Detection Rate of attacks), False Positive Rate (FPR) on benign cases, and average Latency.

B. Results

Table I summarizes the performance.

TABLE I
Intent Verification Performance (n=30)

Model	Recall	Precision	FPR	Latency (s)
grok-4-1-fast	**95.0%**	95.0%	10.0%	**0.99**
claude-haiku-4.5	90.0%	**100%**	**0.0%**	1.64
gemini-flash-lite	85.0%	100%	0.0%	1.36
llama3.2:3b	60.0%	80.0%	30.0%	3.46
gpt-5-nano	15.0%	60.0%	20.0%	6.42

1) Cloud vs. Local Models: The results highlight a significant gap between current local and state-of-the-art cloud models. `grok-4-1-fast` and `claude-haiku` demonstrated superior security reasoning, achieving 90-95% recall. Notably, `grok` was the fastest (0.99s) but had a slight 10% false positive rate, flagging a developer's "search for TODOs" command as suspicious due to its strict interpretation of "execution" tools. In contrast, the local `llama3.2:3b` model achieved only 60% recall and suffered from a 30% false positive rate, struggling with subtle context distinctions. While viable for basic offline protection, it requires further fine-tuning to match cloud verifiers.

2) The Alignment Paradox: An unexpected finding was the poor performance of `gpt-5-nano` (15% recall). Analysis revealed that the model prioritized "Intent Alignment" over "Safety." When a user explicitly asked to "Delete all files," the model verified that the agent's `rm -rf /` action aligned with the user's request and thus marked it as `SAFE`. This underscores a critical vulnerability in Zero Trust design: if the verifier is merely checking if the agent is obedient, it fails to enforce global safety policies.

3) Latency Analysis: Latency remains a bottleneck for real-time agents. While cloud models are approaching sub-second verification, the network round-trip adds overhead. The local model, while private, averaged 3.46s, which may impact user experience in interactive sessions.

C. Threat Model Analysis

We evaluate the security resilience of `llm-cli` against common attack vectors targeting LLM agents. Table II summarizes these mitigations.

TABLE II
Threat Model and Mitigations

Threat Vector	Mitigation in `llm-cli`
Prompt Injection / Jailbreak [14]	**Dynamic Intent Verification**: The secondary LLM detects misalignment between user intent and agent action.
Confused Deputy (Privilege Escalation)	**Workload Identity & RBAC**: Agents have distinct, scoped identities separate from the user.
Tool Squatting / Malicious Registry [13]	**Explicit Registry Verification**: Tools must be explicitly whitelisted by hash or signature, preventing execution of squatted packages.
Remote Code Execution (RCE)	**Scope Verification**: Shell commands are whitelisted and path-restricted. **Global Guardrails** block high-risk binaries.
Log Tampering	**Chained Hashing**: Audit logs are cryptographically linked; modification breaks the chain.
Man-in-the-Middle (MitM)	**RSA/JWT Auth**: Remote MCP connections are authenticated via signed JWTs, preventing spoofing.

VIII. Conclusion

`llm-cli` demonstrates that Zero Trust principles can be effectively applied to AI agents through a combination of cryptographic identity, granular policy enforcement, and semantic analysis. By treating the AI as an untrusted entity and verifying every action explicitly, we can enable powerful agentic workflows while maintaining a robust security posture.

Future work will focus on improving the latency of the intent analyzer through distilled local models and preparing for the quantum era with Post-Quantum Cryptography (PQC) identities.

References

[1] Anthropic, "Model Context Protocol," 2024. [Online]. Available: https://modelcontextprotocol.io/.

[2] Y. Ando, "Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents," *TechRxiv*, 2026. DOI: 10.36227/techrxiv.177006109.97957916/v1.

[3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, 2020.

[4] K. Greshake *et al.*, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv preprint arXiv:2302.12173*, 2023.

[5] Y. Liu *et al.*, "Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study," *arXiv preprint arXiv:2305.13860*, 2023.

[6] OWASP, "OWASP Top 10 for Large Language Model Applications," 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/.

[7] NVIDIA, "NeMo Guardrails," 2023. [Online]. Available: https://github.com/nvidia/NeMo-Guardrails.
[8] L. Pan *et al.*, "Automatically Correcting Large Language Models: Surveying the landscape of diverse self-correction strategies," *arXiv preprint arXiv:2308.03188*, 2023.
[9] B. Beyer *et al.*, "BeyondCorp: A New Approach to Enterprise Security," *Google*, 2014.
[10] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.
[11] H. Chase, "LangChain," 2022. [Online]. Available: https://github.com/langchain-ai/langchain.
[12] Bundesamt für Sicherheit in der Informationstechnik (BSI) and Agence nationale de la sécurité des systèmes d'information (ANSSI), "Design Principles for LLM-based Systems with Zero Trust," 2025.
[13] V. S. Narajala, K. Huang, and I. Habler, "Securing GenAI Multi-Agent Systems Against Tool Squatting: A Zero Trust Registry-Based Approach," *arXiv preprint arXiv:2504.19951*, 2025.
[14] Z. Zhang *et al.*, "Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents," *arXiv preprint arXiv:2503.00061*, 2025.